

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
FALL 2025**



**THE KIOSK GUYS
SMART KIOSK**

**MIGUEL CERVANTES
PRAKASH SAKOPTA
EARL SANCHEZ
THIEN NGUYEN
IVAN KO**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	11.01.2025	MC	Document creation
1.0	11.07.2025	MC, ES, IK	First Version finalized
2.0	03.25.2026	IK	Second Version finalized

CONTENTS

1	Introduction	6
2	System Overview	7
2.1	Presentation Layer Description	7
2.2	Backend Layer Description	7
2.3	Database Layer Description	7
2.4	Hardware Layer Description	7
3	Subsystem Definitions & Data Flow	8
4	Front-End Layer Subsystems	9
4.1	User Login / Registration User Interface	9
4.2	Administrator Dashboard User Interface	10
4.3	Announcement Board User Interface	11
4.4	Supply Inventory User Interface	12
4.5	Interactive Map User Interface	13
4.6	Conference Room Reservation User Interface	14
4.7	API Interactions	15
5	Backend Layer Subsystems	17
5.1	Django	17
5.2	Login Manager	18
5.3	Room Reservation Manager	19
5.4	Supply Order Manager	20
5.5	Automated Email System	21
5.6	Announcement Manager	22
5.7	Admin's Portal Manager	23
6	Database Layer Subsystems	25
6.1	Django ORM	25
6.2	TiDB	26
7	Hardware Layer Subsystems	28
7.1	Touchscreen Monitor	28
7.2	Raspberry Pi 5	29
7.3	WiFi connectivity	30
7.4	Card Swipe Reader	31

LIST OF FIGURES

1	Smart Kiosk data flow diagram	8
2	Front-End Layer Subsystem	9
3	Login / Registration User Interface	10
4	Admin Dashboard User Interface	11
5	Announcement Board User Interface	12
6	Supply Inventory User Interface	12
7	Interactive Map User Interface supported by MappedIn API	13
8	Conference Room Reservation User Interface	14
9	Front-End to Back-End Connection	15
10	Backend Layer subsystem description diagram	17
11	Django Subsystem	17
12	Login Manager Subsystem	18
13	Conference Room Reservation Manager Subsystem	20
14	Supply Order Manager Subsystem	21
15	Automated Email Subsystem	22
16	Announcement Manager Subsystem	23
17	Administrator's Portal Manager Subsystem	24
18	Database subsystem description diagram	25
19	Django ORM Subsystem	25
20	TiDB Subsystem	27
21	Hardware Layer subsystem description diagram	28
22	Touch Screen Monitor Subsystem	28
23	Raspberry Pi 5 Subsystem	29
24	Network Connectivity Subsystem	31
25	Card Swipe Reader Subsystem	32

LIST OF TABLES

2	Subsystem interfaces for User Login/Registration UI	10
3	Subsystem interfaces for Admin Dashboard UI	11
4	Subsystem interfaces for Announcement Board UI	12
5	Subsystem interfaces for Supply Inventory UI	13
6	Subsystem interfaces for Interactive Map UI	14
7	Subsystem interfaces for Room Reservation UI	15
8	Subsystem interfaces for API Interactions	16
9	Subsystem interfaces for Django	18
10	Subsystem interfaces for Login Manager	19
11	Subsystem interfaces for Room Reservation Manager	20
12	Subsystem interfaces for Supply Order Manager	21
13	Subsystem interfaces for Automated Email System	22
14	Subsystem interfaces for Announcement Manager	23
15	Subsystem interfaces for Admin's Portal Manager	24
16	Subsystem interfaces for Django ORM	26
17	TiDB interfaces	27
18	Subsystem interfaces for Touchscreen	29
19	Subsystem interfaces for Raspberry Pi 5	30
20	Subsystem interfaces for WiFi connectivity	31

21	Subsystem interfaces for Card Swiper Reader	32
----	---	----

1 INTRODUCTION

The Smart Kiosk System is a web based service designed to improve convenience and accessibility for UTA staff and students. The system includes one kiosk located in the lobby of the Engineering Research South A (ERSA) building, two kiosks at the main entrances of the Engineering Research Building (ERB), and a remote website that can be accessed from personal devices. Each kiosk is tailored to the needs of its users, whether they are faculty and researchers in ERSA or students and visitors in ERB.

The Smart Kiosk is a standalone device powered by a Raspberry Pi 5 running in kiosk mode, which restricts users to only accessing the Smart Kiosk website. The website uses React (via Vite) for the front-end interface and communicates with a Django backend through REST APIs. The backend connects to a TiDB database to store and manage system data. The entire system is hosted on Render to ensure reliability and security.

The system provides different features depending on the building. In ERB, the kiosk offers an interactive 3D map of the building with room search and navigation, along with a slideshow panel for announcements and events. In ERSA, the kiosk includes these same features, with additional functionality for room reservations and supply ordering. A login system is used to identify users, and an admin feature allows authorized users to manage system data and settings. The remote website is limited to room reservations and administrative functions to encourage use of the physical kiosks.

Each kiosk is designed with its primary users in mind. The ERSA kiosk is mainly used by faculty and research staff, so login is required to access features like conference room reservations and supply requests. Guests without an account can still use the interactive map and view announcements but must register to access additional features. In contrast, the ERB kiosks are primarily used by students and visitors, so they only provide the interactive map and announcement display. Login is available only for administrators to perform maintenance tasks. The remote website is intended for ERSA users, allowing them to reserve rooms and manage administrative settings.

2 SYSTEM OVERVIEW

This section describes the overall structure of the Smart Kiosk and its software system. Our team has decided to utilize the widely used software pattern known as the "Layered Architecture" to organize and build our device. The system will consist of seven main layers that all communicate and help one another to achieve the main requirements of the Smart Kiosk

2.1 PRESENTATION LAYER DESCRIPTION

The Presentation Layer, also referred to as the front-end of the web application, is what the end-user sees and interacts with on the Smart Kiosk. Our team has placed an emphasis on making sure this layer provides an interface that is aesthetically pleasing, user-friendly, and representative of UTA's culture. The front-end will achieve this by using libraries and web frameworks such as React.js, JSX, and CSS to create a sleek and navigable graphical user interface that displays information and captures user input. Some components of this layer include the user interfaces for login and registration, room reservation feature, supply request feature, ERSA mapping functionality, and the home page. Finally, this layer exclusively communicates with the backend through APIs in a client-server relationship to ensure user requests are communicated properly. This system is deployed on Render.

2.2 BACKEND LAYER DESCRIPTION

The Backend layer of the Smart Kiosk is responsible for managing the business and server-side logic of the web application. This includes some of the core functionalities of the Smart Kiosk such as room reservations, supply requests and the ability to create user profiles. All operations in this layer are performed behind the scenes and are invisible to end-users. The Smart Kiosk's backend is implemented using Django, a Python web framework, to efficiently manage user authentication, authorization, and email handling. REST APIs will be implemented through Django to allow for communication with the front-end and database layers. Some examples include integrating Microsoft Outlook and MappedIn. These APIs act as the middleman, and ensure the integrity and security of the data such as user input and client requests. This system is deployed on Render.

2.3 DATABASE LAYER DESCRIPTION

The Database Layer of the Smart Kiosk manages how the backend stores and retrieves data. This layer will utilize TiDB, a SQL database, to store information such as user profiles, room reservations, supplies and their requests, among other data. The backend will be able to access this database through Django Object-Relational Mapper (ORM). Django ORM abstracts database information and acts as a translator of Python to SQL queries, allowing for efficient and secure communication between the two layers.

2.4 HARDWARE LAYER DESCRIPTION

The Hardware Layer of the Smart Kiosk encompasses all the physical components that end-users can touch and see, and support the operation of the web application. Its functionality is to serve as a platform where visitors of the ERSA building can access the web applications functionalities through a large, sturdy device. The features of this layer include an 18.5-inch touchscreen display, a Raspberry Pi 5 single-board computer, WiFi connectivity, a card swiper reader, and a secure stand. These features enhance the accessibility, computing power, and foundation of the Smart Kiosk.

3 SUBSYSTEM DEFINITIONS & DATA FLOW

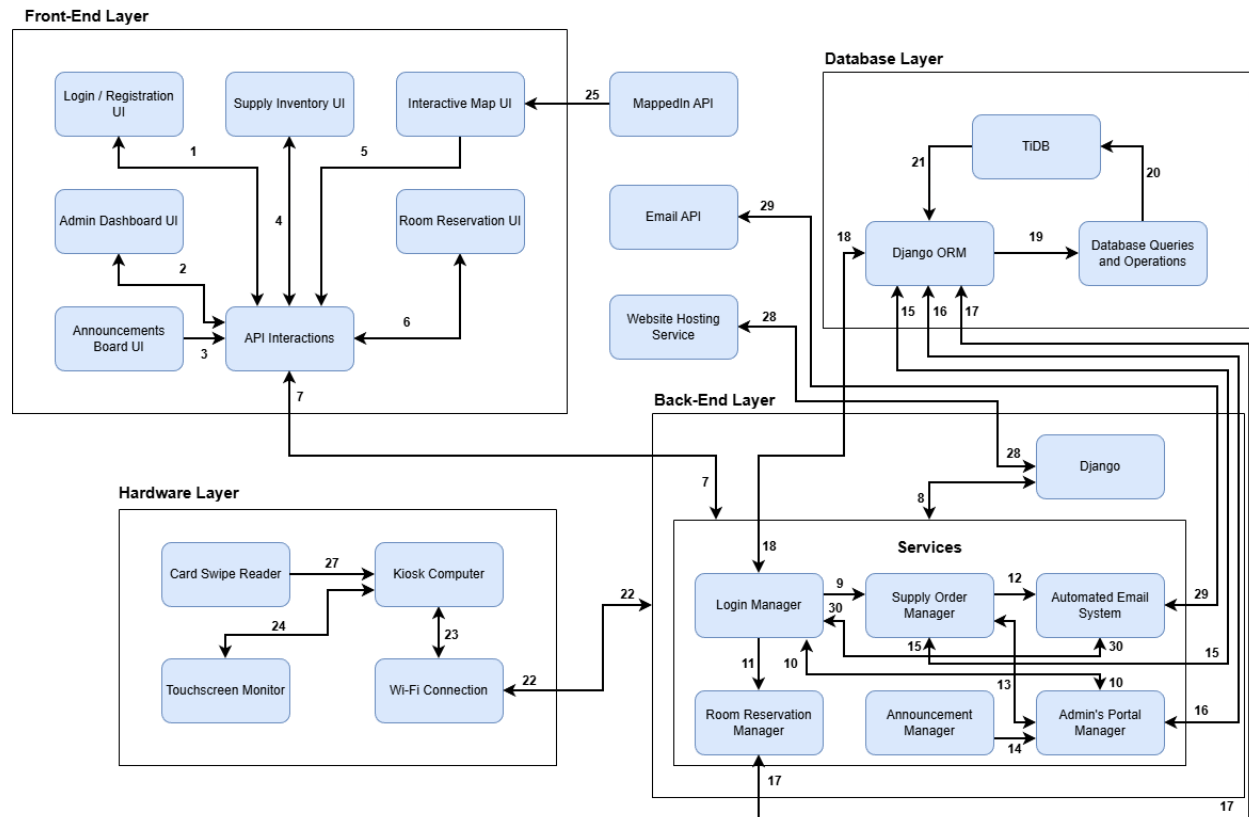


Figure 1: Smart Kiosk data flow diagram

4 FRONT-END LAYER SUBSYSTEMS

The Front-End Layer is the portion of the Smart Kiosk web application with which end users see and interact. It renders the user interface, captures input, and exchanges data with the Back-End Layer through React-based JavaScript logic. This section describes the layer's subsystems, their responsibilities, and how they collaborate to deliver a responsive, accessible kiosk experience.

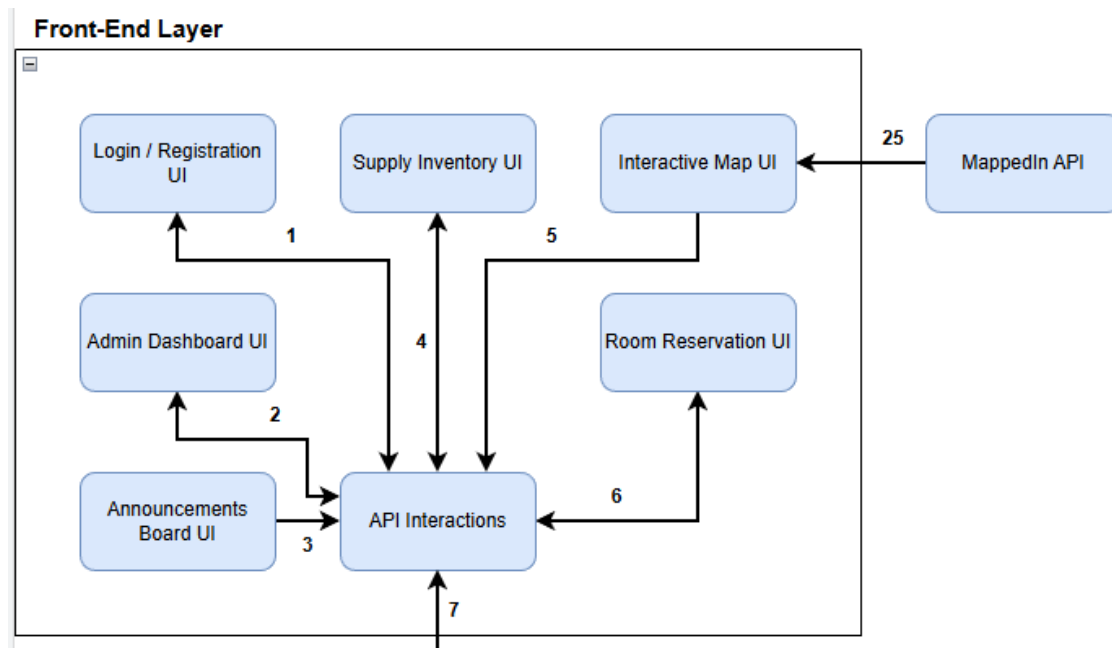


Figure 2: Front-End Layer Subsystem

4.1 USER LOGIN / REGISTRATION USER INTERFACE

The Login and Registration User Interface is the initial screen presented to users when the Smart Kiosk system is launched. From this screen, users can create a new account (registration) or log in as a returning user (authentication). Once the user's credentials are successfully verified by back-end operations, the system grants access to additional kiosk functions beyond the publicly available interactive building map, such as the room reservation and supply ordering.

4.1.1 ASSUMPTIONS

- Assumes a secure connection to the back-end system.
- Assumes that the backend is responsible for validating the login data to ensure secure logins.

4.1.2 RESPONSIBILITIES

- Facilitates the user interface for taking in users' login data to be registered as a new user, or to log in as a returning user.

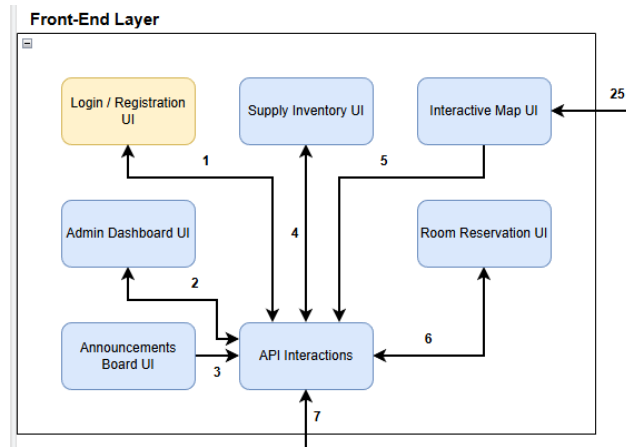


Figure 3: Login / Registration User Interface

4.1.3 SUBSYSTEM INTERFACES

Table 2: Subsystem interfaces for User Login/Registration UI

ID	Description	Inputs	Outputs
#1	User's credentials input boxes	Username Password Email	Login Request
#1	Login Status	Back-End Authentication Data	Change to Home Page if login success

4.2 ADMINISTRATOR DASHBOARD USER INTERFACE

The Admin Dashboard User Interface provides authorized administrative users with a centralized control panel for managing the Smart Kiosk system. Through this interface, administrators can view system status, kiosk usage metrics, manage user accounts, update building announcements or room availability data. Administrative actions performed through this UI are securely transmitted to the backend, ensuring that only authorized individuals can modify system data or configure settings. This interface is designed to streamline maintenance and oversight of the kiosk while maintaining system reliability and security.

4.2.1 ASSUMPTIONS

- The administrator has valid credentials and is authenticated by the back-end before accessing the Admin UI.
- Assumes a secure connection to the back-end system.
- Any changes to the kiosk saved by the admin user propagate consistently across all kiosk application instances.

4.2.2 RESPONSIBILITIES

- Display real time system status data in a clear and organized manner.

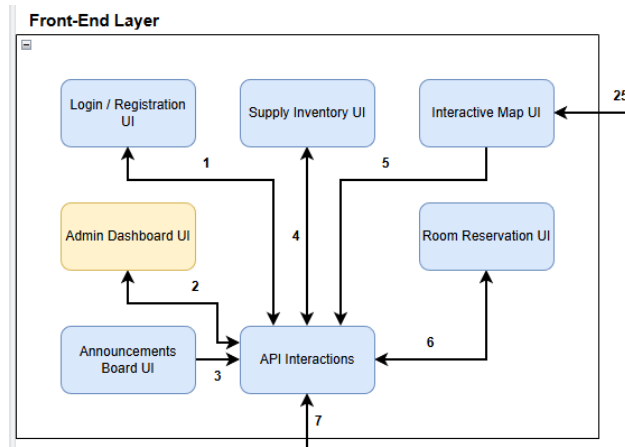


Figure 4: Admin Dashboard User Interface

- Allow admin users to create, modify, and publish building announcements.
- Offer tools to modify the room reservation, supply ordering, and interactive map features.

4.2.3 SUBSYSTEM INTERFACES

Table 3: Subsystem interfaces for Admin Dashboard UI

ID	Description	Inputs	Outputs
#2	Admin Dashboard	Data change/add to functions	Updated functions

4.3 ANNOUNCEMENT BOARD USER INTERFACE

The Announcement Board UI presents building updates, alerts, and general information in a slideshow-style display, automatically cycling through announcements at a fixed interval. The slideshow updates dynamically when new announcements are added through the administrative dashboard, ensuring that users always see the most current information. This interface is designed for simplicity and readability, allowing users to quickly absorb key messages without needing to interact with the screen.

4.3.1 ASSUMPTIONS

- Admins have already created and scheduled announcements through the admin dashboard.
- Assumes a secure connection to the back-end system.
- Users are expected to view announcements with no interaction required on this screen.

4.3.2 RESPONSIBILITIES

- Display announcements in a slideshow format.
- Refresh automatically whenever announcements are updated or added.

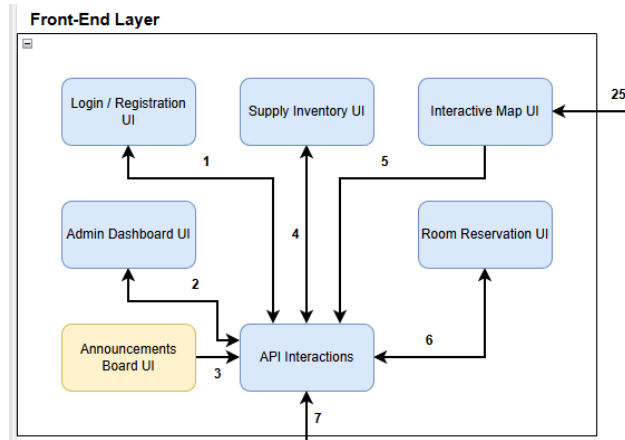


Figure 5: Announcement Board User Interface

4.3.3 SUBSYSTEM INTERFACES

Table 4: Subsystem interfaces for Announcement Board UI

ID	Description	Inputs	Outputs
#3	Announcement Slideshow Interface	N/A	Announcement Posts

4.4 SUPPLY INVENTORY USER INTERFACE

The Supply Inventory UI allows users to report supplies that are low or depleted within the ERSA building. Items are organized into categories, with frequently requested or commonly depleted items automatically displayed at the top for faster access. Users simply tap or select the item that needs restocking, and the system sends a low-supply notification to the backend where an email is drafted for the restocker or administrator to review and schedule replenishment. The interface is designed for simplicity and efficiency, minimizing the number of steps required for users to submit supply requests.

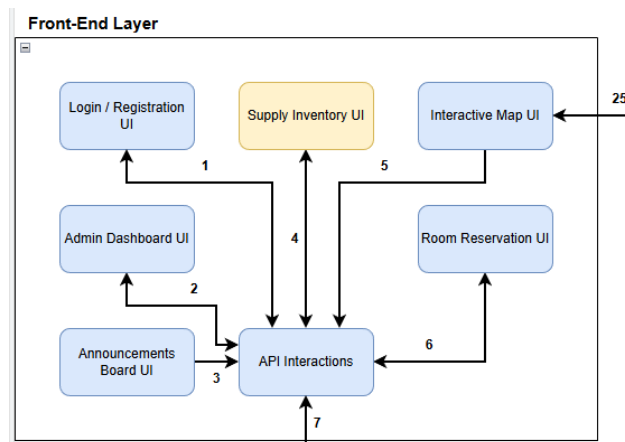


Figure 6: Supply Inventory User Interface

4.4.1 ASSUMPTIONS

- Requires users to login.
- Assumes a secure connection to the back-end system.
- Users are familiar with supplies commonly found in the building.

4.4.2 RESPONSIBILITIES

- Display supplies in an organized, category based layout.
- Frequently requested items shown at the top of each category.
- Easily request items by tapping on supply images and tapping request button after selection.
- Send a request to the backend to finish restocking process.

4.4.3 SUBSYSTEM INTERFACES

Table 5: Subsystem interfaces for Supply Inventory UI

ID	Description	Inputs	Outputs
#4	Supply Inventory Panel	Item Selections	Restock Request
#4	Supply Inventory Panel	Back-end Supply Data	Updated UI

4.5 INTERACTIVE MAP USER INTERFACE

The Interactive Map UI embeds a 3D map of the building, a service provided by MappedIn, allowing users to search for destinations and receive a highlighted path from the kiosk location. The interface supports pan/zoom, floor switching, and room selection, then renders a clear 3D path with markers and turn prompts. It is optimized for kiosk readability and listens for map inputs and context without leaving the kiosk application.

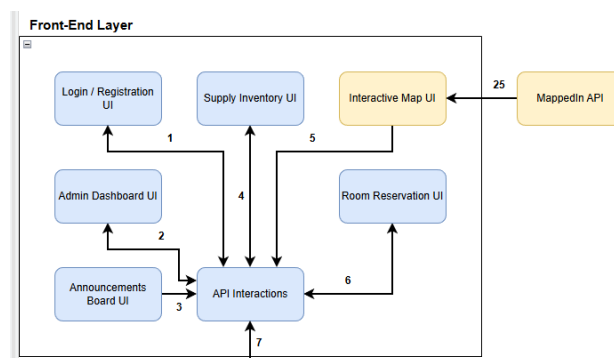


Figure 7: Interactive Map User Interface supported by MappedIn API

4.5.1 ASSUMPTIONS

- Building's layout and room data are already provided to MappedIn.
- Assumes a secure connection to the network to access MappedIn.
- Kiosk location is predefined for easy routing.
- Map interactions are provided by MappedIn.

4.5.2 RESPONSIBILITIES

- Initialize and display embedded MappedIn 3D map.
- Provide room search, selection, and accessible routes.

4.5.3 SUBSYSTEM INTERFACES

Table 6: Subsystem interfaces for Interactive Map UI

ID	Description	Inputs	Outputs
#5	MappedIn Interfaces	Map Interactions	Request to MappedIn API
#25	MappedIn API	Map Requests	Requested Map Data

4.6 CONFERENCE ROOM RESERVATION USER INTERFACE

The Conference Room Reservation UI lets users find and book available rooms. The UI validates conflicts in real time, confirms the booking with a clear summary, and sends a request to the back-end to complete the room reservation process. Designed for quick, walk-up use, it emphasizes clear availability indicators and minimal steps to complete a reservation.

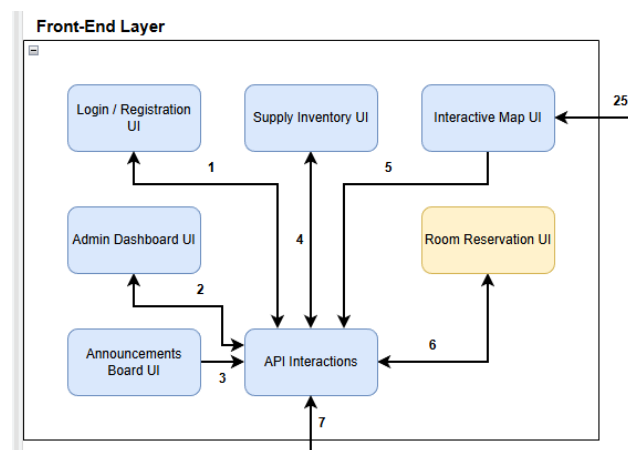


Figure 8: Conference Room Reservation User Interface

4.6.1 ASSUMPTIONS

- Room data is maintained by administrators.
- Assumes a secure connection to the back-end system.
- Requires user to sign in.

4.6.2 RESPONSIBILITIES

- Display real-time availability with conflict detection and prevention.
- Show clear success/error states and provide confirmation screen.

4.6.3 SUBSYSTEM INTERFACES

Table 7: Subsystem interfaces for Room Reservation UI

ID	Description	Inputs	Outputs
#6	Room Reservation Panel	Room Selection, Time, User Data	Room Reservation Request
#6	Room Reservation Panel	Back-End Room Reservation Data	Updated Room Information

4.7 API INTERACTIONS

The application's API layer enables React components to exchange data with the Django backend over authenticated REST/JSON endpoints. This allows for reliable data transmission between the front-end UI elements and the back-end processes logic.

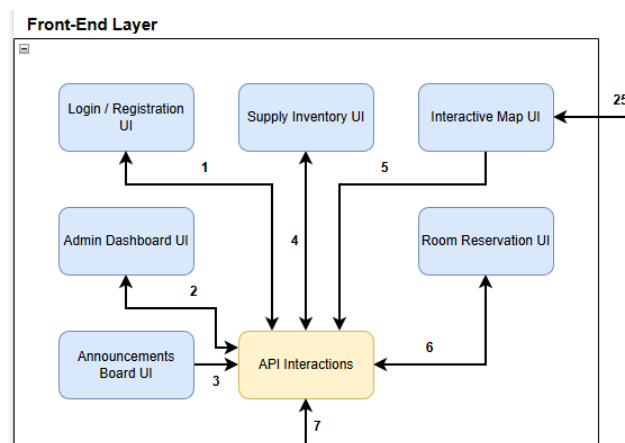


Figure 9: Front-End to Back-End Connection

4.7.1 ASSUMPTIONS

- Stable network
- Assumes a secure connection to the back-end system.

4.7.2 RESPONSIBILITIES

- Provide essential operations to send front-end data to the back-end system and vice versa.

4.7.3 SUBSYSTEM INTERFACES

Table 8: Subsystem interfaces for API Interactions

ID	Description	Inputs	Outputs
#7	API Interactions	Front-End/Back-End Data	Back-End/Front-End Data

5 BACKEND LAYER SUBSYSTEMS

The Backend Layer consists of the core components working behind the scenes to ensure the Smart Kiosk functions properly and displays appropriate data to users.

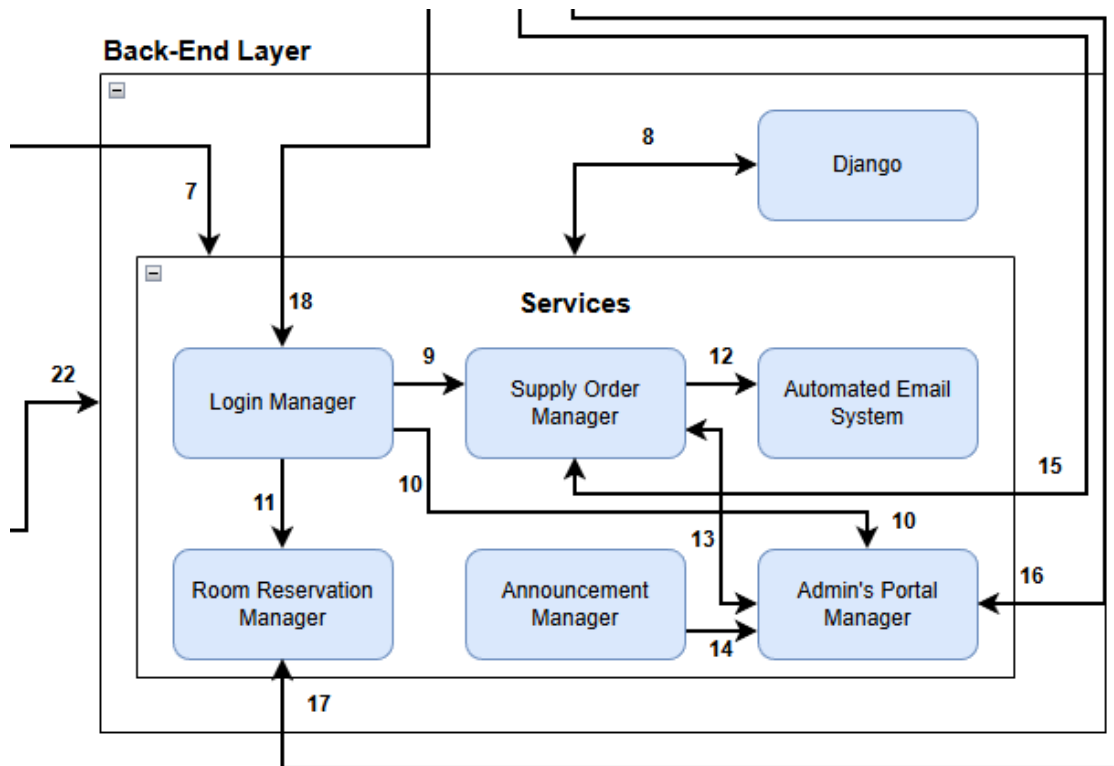


Figure 10: Backend Layer subsystem description diagram

5.1 DJANGO

The Django service houses all the key managers and processing units. Its primary function is to handle the flow of information and execute the system's business logic.

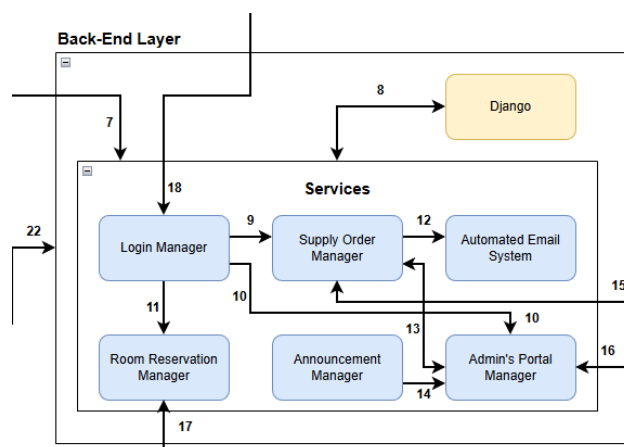


Figure 11: Django Subsystem

5.1.1 ASSUMPTIONS

Our team assumes that the services that are being provided through the use of Django function properly. We assume that Django itself does not contain any issues or poses any potential security risks that could affect the integrity of the kiosk's website.

5.1.2 RESPONSIBILITIES

Django is responsible for providing the frameworks in which our backend services will run on. The data flow works through Django providing the framework for the backend code which provides the services that the users see displayed on the website.

5.1.3 SUBSYSTEM INTERFACES

Table 9: Subsystem interfaces for Django

ID	Description	Inputs	Outputs
#8	Data flow from the Services subsystem, representing calls to core Django functionality (e.g., ORM, routing, request/response handling).	Service Call Request Data Database Queries	API Responses Query Results Framework Services
#7	Data flow into the Login Manager, representing the initial request handling and routing from the outside world.	HTTP Requests URL Routing Data	N/A

5.2 LOGIN MANAGER

The Login Manager service ensures that users are able to sign up and login in order to access the kiosk's functionality

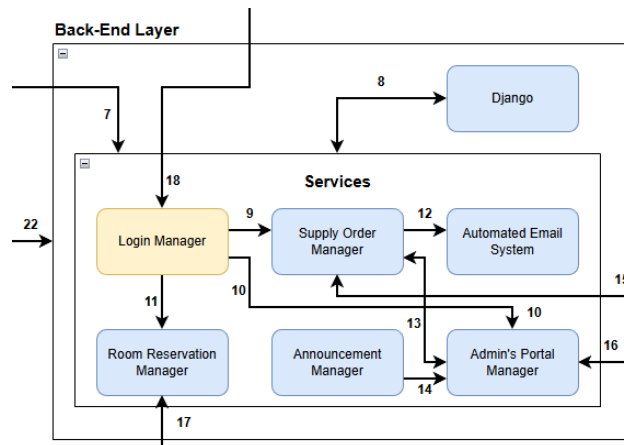


Figure 12: Login Manager Subsystem

5.2.1 ASSUMPTIONS

Our team assumes that the users that will be utilizing the service are UTA staff and students, so that only UTA emails will be allowed to function.

5.2.2 RESPONSIBILITIES

The Login Manager will be responsible for ensuring that users are able to login to view their reservations, request supplies, and view announcements as well. This will also ensure that admins are provided an admin view when they login.

5.2.3 SUBSYSTEM INTERFACES

Table 10: Subsystem interfaces for Login Manager

ID	Description	Inputs	Outputs
#22	External input from the Kiosk/Frontend layer, representing user credentials during the login attempt.	Username (UTA Email) Password Session Request	Authentication Status
#18	Input from the Backend Layer boundary (likely a route from Django), representing the initial request to the service.	HTTP Request Routing Data	N/A
#11	Output to the Room Reservation Manager, granting access or providing user context for booking actions.	User/Session ID Access Token	N/A
#9	Output to the Supply Order Manager, granting access or user context for making supply requests.	User ID Authorization Status	N/A

5.3 ROOM RESERVATION MANAGER

The Room Reservation Manager is a core service responsible for managing the allocation and scheduling of rooms. It handles requests for new reservations, modifications to existing reservations, and provides real-time availability information.

5.3.1 ASSUMPTIONS

Our team assumes that the Room Reservation Manager receives accurate and validated user data (e.g., login credentials from the Login Manager) to process requests. We also assume a reliable data store exists to maintain current room availability and reservation records.

5.3.2 RESPONSIBILITIES

The Room Reservation Manager is responsible for processing room booking requests, verifying room availability, and logging the reservation details. It communicates with the Login Manager to ensure only authorized users can make bookings.

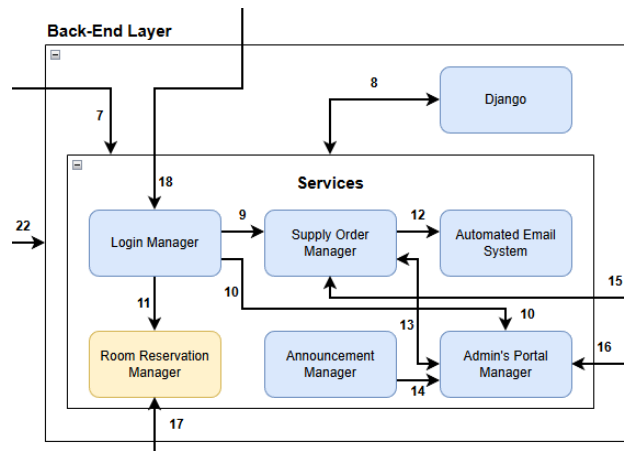


Figure 13: Conference Room Reservation Manager Subsystem

5.3.3 SUBSYSTEM INTERFACES

Table 11: Subsystem interfaces for Room Reservation Manager

ID	Description	Inputs	Outputs
#11	Data flow from the Login Manager, granting access after successful authentication.	User/Session ID User Role Data	N/A
#17	External input from the Kiosk/Frontend layer, representing user requests for room booking.	Reservation Request Search Criteria	Booking Confirmation Availability Status

5.4 SUPPLY ORDER MANAGER

The Supply Order Manager is the service that oversees the process of ordering supplies for the facility. It receives requests for supplies, processes them, and initiates communication for fulfillment and tracking.

5.4.1 ASSUMPTIONS

We assume that supply requests received are properly formatted and contain necessary details (e.g., item, quantity, location). We also assume a reliable connection to external ordering systems or inventory databases is available for processing.

5.4.2 RESPONSIBILITIES

The Supply Order Manager is responsible for validating supply requests, maintaining an order log, and triggering the Automated Email System to notify relevant parties about the order status. It also provides order information to the Admin's Portal Manager.

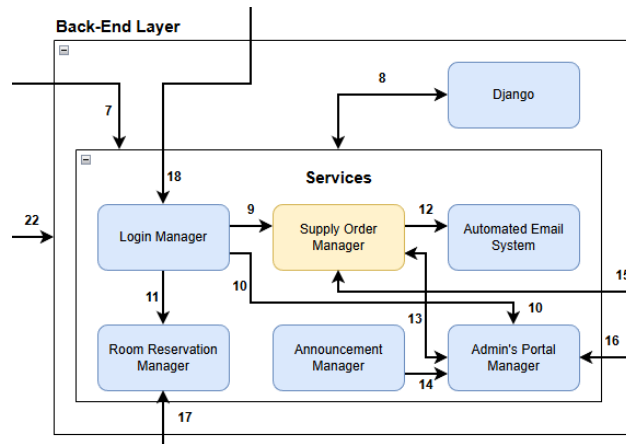


Figure 14: Supply Order Manager Subsystem

5.4.3 SUBSYSTEM INTERFACES

Table 12: Subsystem interfaces for Supply Order Manager

ID	Description	Inputs	Outputs
#9	Input from the Login Manager, likely a request for a new supply order from an authenticated user.	Supply Order Details User ID	N/A
#12	Output to the Automated Email System, instructing it to send a notification regarding the order.	Order Status Recipient Email Order Summary	N/A
#10	Output to the Admin's Portal Manager, providing order details for administrative review and action.	Pending Orders List Order ID/Status	N/A

5.5 AUTOMATED EMAIL SYSTEM

The Automated Email System is a utility service focused solely on sending predefined email notifications triggered by other backend services, such as confirmations or status updates. It operates using Django SMTP and a dedicated G-mail account for the kiosk.

5.5.1 ASSUMPTIONS

Our team assumes that this system is correctly configured with SMTP (Simple Mail Transfer Protocol) credentials and that the email templates are properly defined. We assume the system successfully delivers emails based on the instructions it receives.

5.5.2 RESPONSIBILITIES

The Automated Email System is responsible for generating and dispatching emails to designated recipients based on inputs received from other managers, primarily the Supply Order Manager. It serves as

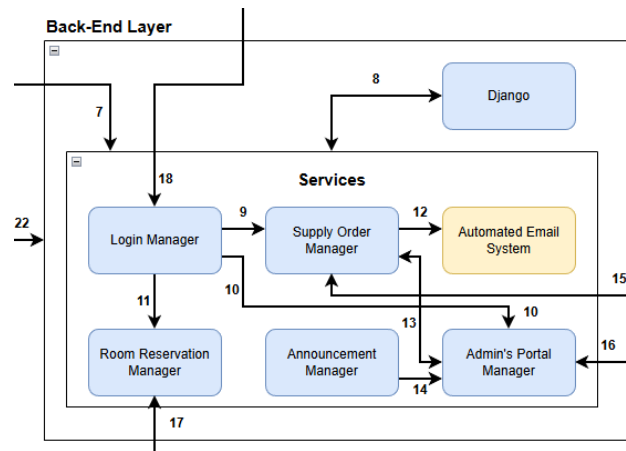


Figure 15: Automated Email Subsystem

the primary communication bridge for system-generated notifications.

5.5.3 SUBSYSTEM INTERFACES

Table 13: Subsystem interfaces for Automated Email System

ID	Description	Inputs	Outputs
#12	Input from the Supply Order Manager, containing the details needed to construct and send an order-related email.	Order Status Recipient Email Email Body Content	N/A
#15	External output to the email server, representing the final delivery of the email to the recipient.	N/A	Email Message

5.6 ANNOUNCEMENT MANAGER

The Announcement Manager is a service dedicated to managing and publishing system-wide announcements or notices for users. It controls what messages or promotional items are displayed to which users. Admin users can upload or change banner displays through the physical kiosk systems or the remote site.

5.6.1 ASSUMPTIONS

We assume that only authorized personnel (likely via the Admin's Portal Manager) can input, modify, or remove announcements. We also assume a mechanism exists in the frontend to correctly display the announcements retrieved by this manager.

5.6.2 RESPONSIBILITIES

The Announcement Manager is responsible for storing, retrieving, and managing the lifecycle of public announcements. It provides the Admin's Portal Manager with the interface to manage the announcement data.

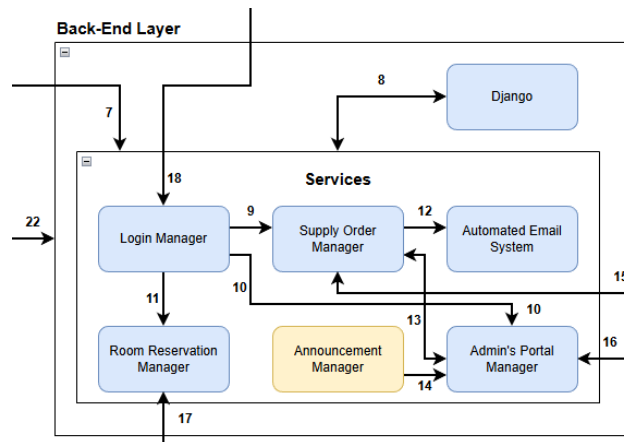


Figure 16: Announcement Manager Subsystem

5.6.3 SUBSYSTEM INTERFACES

Table 14: Subsystem interfaces for Announcement Manager

ID	Description	Inputs	Outputs
#14	Input from the Admin's Portal Manager, used to create, edit, or delete announcements.	Announcement Content Start/End Date Priority Level	N/A
#13	Output to the Admin's Portal Manager, providing a list of current announcements for administrative review.	N/A	Current Announcements List

5.7 ADMIN'S PORTAL MANAGER

The Admin's Portal Manager serves as the main interface and logic handler for administrative functions, aggregating information from and sending control signals to various other services.

5.7.1 ASSUMPTIONS

We assume that access to the Admin's Portal Manager is strictly limited to users with the highest authorization levels, validated by the Login Manager. We also assume administrative actions, such as managing orders or announcements, are logged for auditing purposes.

5.7.2 RESPONSIBILITIES

The Admin's Portal Manager is responsible for aggregating pending supply order details (from the Supply Order Manager), managing announcements (via the Announcement Manager), and handling other high-level administrative tasks. It manages the output to the external administrative interface.

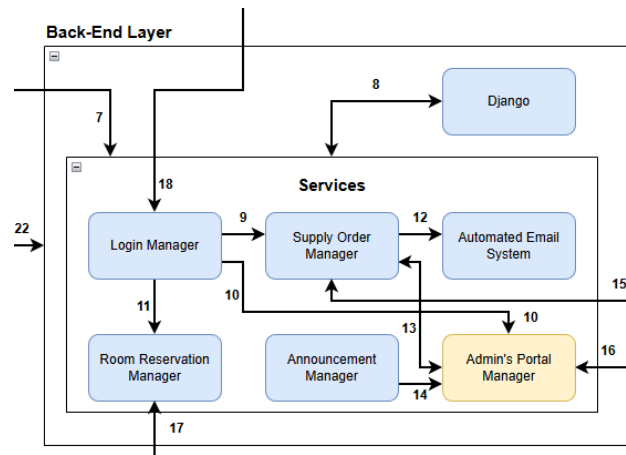


Figure 17: Administrator's Portal Manager Subsystem

5.7.3 SUBSYSTEM INTERFACES

Table 15: Subsystem interfaces for Admin's Portal Manager

ID	Description	Inputs	Outputs
#10	Input from the Supply Order Manager, providing details on new or pending supply orders that require administrative attention.	Order ID Status Update Order Details	N/A
#13	Input from the Announcement Manager, providing a list of current announcements for editing/review.	Current Announcements List	N/A
#14	Output to the Announcement Manager, sending new/updated announcement data.	N/A	Announcement Data
#16	External output to the administrative interface/frontend, providing the necessary data for display.	N/A	Portal View Data Order Action Status

6 DATABASE LAYER SUBSYSTEMS

The database layer is where and how the backend manages data for the Smart Kiosk's web application. This section will go through the key subsystems that make up this layer.

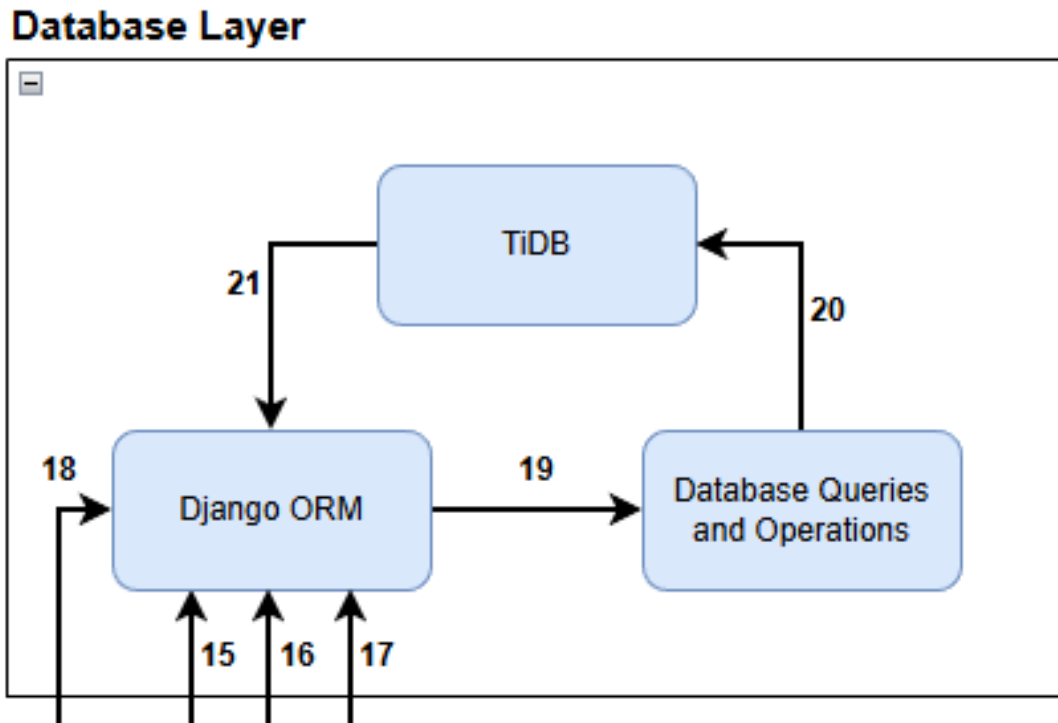


Figure 18: Database subsystem description diagram

6.1 DJANGO ORM

The Django ORM subsystem works with the backend to manage how it accesses and edits data in the TiDB database.

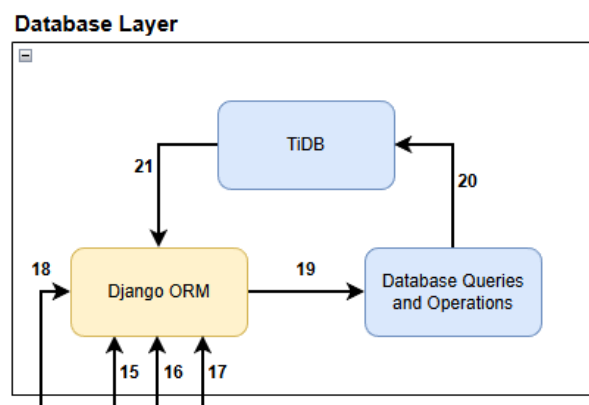


Figure 19: Django ORM Subsystem

6.1.1 ASSUMPTIONS

Our team assumes that Django ORM will perform its CRUD operations efficiently to match our sponsor's requirements. Our team will also assume that those operations will be compatible and safe with the TiDB database to protect from threats such as SQL injections.

6.1.2 RESPONSIBILITIES

Django ORM is responsible for handling the logic behind the main CRUD operations and maintaining their relationships by abstracting the database information and acting as a translator between Python objects to SQL queries. The data flow of this subsystem operates by receiving data from the backend, translating it, and storing it in the TiDB database.

6.1.3 SUBSYSTEM INTERFACES

Table 16: Subsystem interfaces for Django ORM

ID	Description	Inputs	Outputs
#1	Data flow from the Backend Services (e.g., Login Manager, Supply Order Manager) requesting data operations.	Python Objects CRUD Commands Filtering Criteria	N/A
#2	Data flow to the TiDB database, representing the translated, executable SQL query.	N/A	SQL Queries (SELECT, INSERT, UPDATE, DELETE)
#3	Data flow from the TiDB database, returning raw query results.	Raw Query Results Status Codes	N/A
#4	Data flow to the Backend Services, representing the results of the operation, translated back into Python objects.	N/A	Model Instances (Python Objects) Success/Failure Status

6.2 TiDB

TiDB is the platform that the Smart Kiosk will store the users and its own application data. The database is open-source and made up of relational SQL-based tables.

6.2.1 ASSUMPTIONS

Our team assumes that the TiDB database will provide sufficient scalability, availability, and storage to meet the users' demands. Our team also assumes that only authorized personnel will be able to modify the database.

6.2.2 RESPONSIBILITIES

This subsystem is responsible for being a secure and accessible location for where the Smart Kiosk stores its data. It interacts directly with Django ORM to receive and store data such as user profiles, room reservations, and announcements.

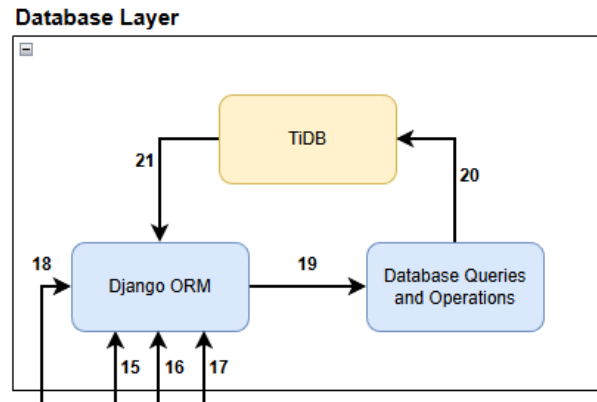


Figure 20: TiDB Subsystem

6.2.3 SUBSYSTEM INTERFACES

Table 17: TiDB interfaces

ID	Description	Inputs	Outputs
#2	Data flow from the Django ORM, consisting of SQL commands to be executed.	SQL Queries (DDL, DML) Transaction Commands	N/A
#3	Data flow to the Django ORM, returning the results of the executed SQL query.	N/A	Requested Data Rows Success/Error Codes

7 HARDWARE LAYER SUBSYSTEMS

The Hardware Layer encapsulates the Smart Kiosk's web application and serves as the physical interface for visitors of the ERSA building to interact with. This section will go through the key subsystems that make up this layer.

Hardware Layer

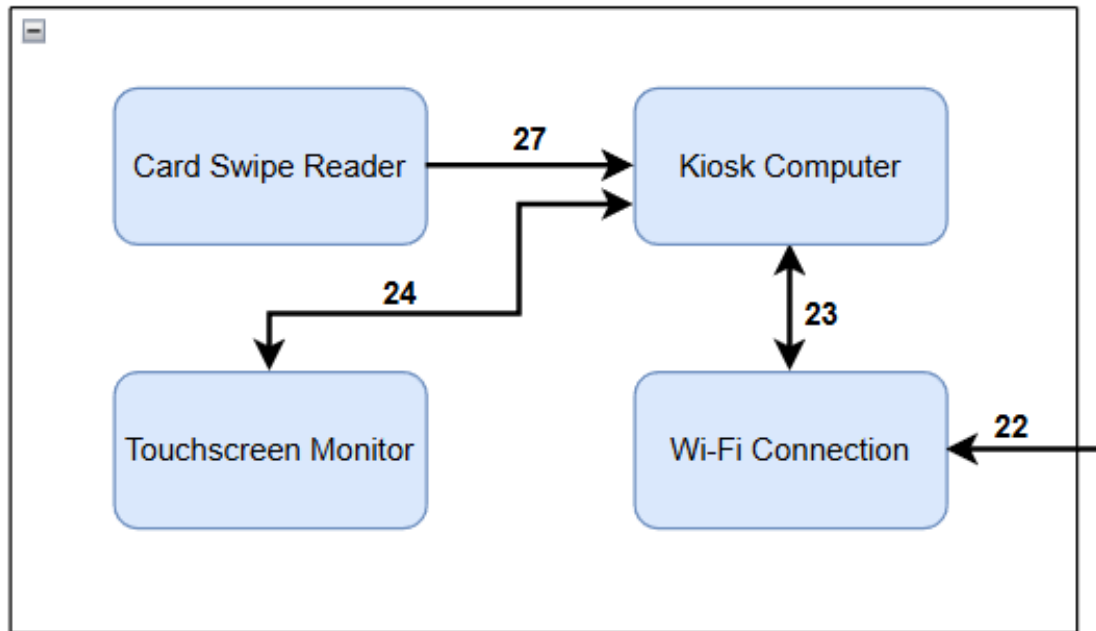


Figure 21: Hardware Layer subsystem description diagram

7.1 TOUCHSCREEN MONITOR

The touchscreen monitor is the primary interface through which end-users physically interact with the Smart Kiosk to access the web application. It communicates directly with the Presentation Layer to display a clear, visually engaging, and responsive graphical user interface.

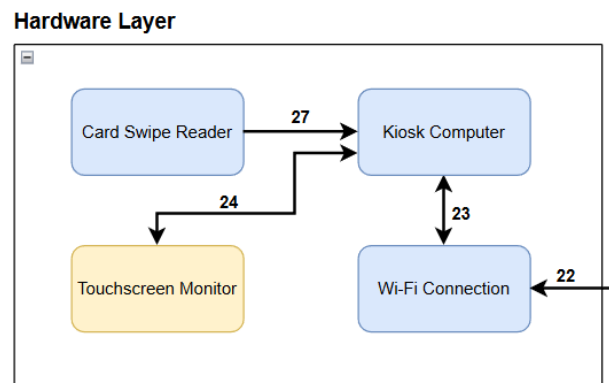


Figure 22: Touch Screen Monitor Subsystem

7.1.1 ASSUMPTIONS

Our team assumes that the touchscreen monitor will display the front-end of our web application at a 1080P resolution. We also assume that our display will be able to support 10 point multi-touch functionality and will be able to interact with the front-ends components without any noticeable input delay.

7.1.2 RESPONSIBILITIES

This subsystem of the hardware layer is responsible for displaying the web application at high resolution and accurately capturing user interactions instantaneously. The monitors touchscreen functionalities allow for direct communication between the user and web application, removing the need for traditional input devices such as a mouse and keyboard.

7.1.3 SUBSYSTEM INTERFACES

Table 18: Subsystem interfaces for Touchscreen

ID	Description	Inputs	Outputs
#24	The monitor displays the web application	data being displayed	N/A
#24	Monitor captures user touch interactions and sends them to the Raspberry Pi 5 for data processing.	User touch interactions	Sends touch data to RPI 5

7.2 RASPBERRY PI 5

The Raspberry Pi 5 is the computer that the Smart Kiosk will use to power the device. It communicates with the other subsystems in the hardware layer to efficiently and safely run the web application.

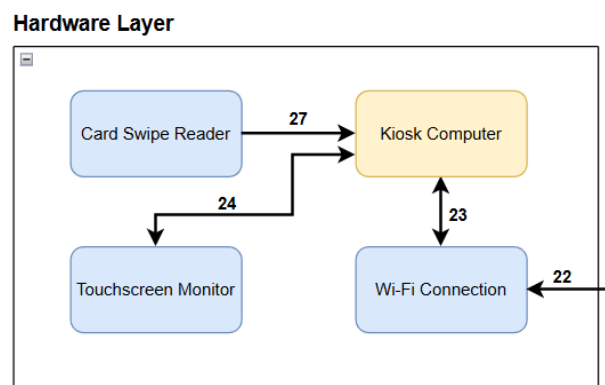


Figure 23: Raspberry Pi 5 Subsystem

7.2.1 ASSUMPTIONS

Our team assumes that the Raspberry Pi 5 will have efficient power management to protect the device from overheating and power supply issues. We also assume that the device will have enough computing

power and storage to run the software of our web application smoothly. Our team also assumes that the operating system and WiFi chip of the Raspberry Pi will be compatible with our web application.

7.2.2 RESPONSIBILITIES

This subsystem of the hardware layer is responsible for being the brain and central controller of the Smart Kiosk as it executes the Smart Kiosk's web application on its operating system. The Raspberry Pi 5 is responsible for managing its resources efficiently in order to handle any intensive load being rendered and entered by the end-user.

7.2.3 SUBSYSTEM INTERFACES FOR RASPBERRY PI 5

Table 19: Subsystem interfaces for Raspberry Pi 5

ID	Description	Inputs	Outputs
#23	The RPI 5 sends and receives data through its connection to the WiFi to the web applications backend.	User and Kiosk Data N/A	N/A Web and API requests
#24	The RPI 5 receives and processes the user touch data and updates the state of the web application. After that it sends the new graphical display data for the monitor to display	User touch input from Responses from the backend	Updated UI pages The new data to display on the monitor
#27	The RPI 5 receives and processes the magnetic card swipe information	Magnetic Card Swiper Data	Data is sent to the backend

7.3 WiFi CONNECTIVITY

WiFi is the wireless networking connectivity method that the Smart Kiosk will use to connect the device to the internet. The Raspberry Pi 5 used to power the system will connect to the ERSA buildings router through a wireless connection and use common network protocols such as IP, TCP, and HTTP to access the internet.

7.3.1 ASSUMPTIONS

Our team assumes that ERSA building will connect to the Smart Kiosk seamlessly and stay connected whenever the device is powered on at a 99.99 percent rate. Our team assumes that the network which it is connected to will be a private and secure network. Our team assumes that the people who oversee the ERSA building will ensure that the WiFi connection is working properly.

7.3.2 RESPONSIBILITIES

This subsystem is responsible for connecting the Smart Kiosk to the internet in order for the device to be able to access its web application. It is in charge of maintaining a stable connection with the internet to avoid any downtime problems to help facilitate backend services.

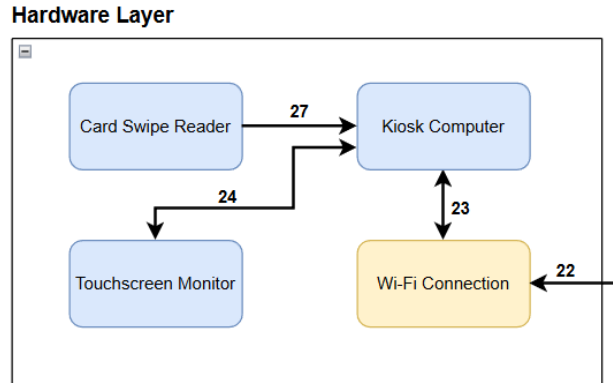


Figure 24: Network Connectivity Subsystem

7.3.3 SUBSYSTEM INTERFACES

Table 20: Subsystem interfaces for WiFi connectivity

ID	Description	Inputs	Outputs
#22	WiFi connects to ERSAs router to gain access to the internet so that data can be exchanged from the Smart Kiosk to the internet	WiFi connection & Data collected from RPI 5	Data sent from RPI 5
#23	WiFi receives and delivers web requests and packets from the Raspberry Pi	N/A	Raspberry Pi receives corresponding packets and responses from WiFi

7.4 CARD SWIPE READER

The card swipe reader is an external device that connects to the kiosk via USB. It allows users to log in by swiping their UTA ID cards instead of manually entering their credentials. The reader scans the magnetic stripe on the card and sends the encoded identification data to the backend system for processing.

7.4.1 ASSUMPTIONS

The card swipe reader assumes that users possess valid UTA ID cards with properly functioning magnetic stripes and that the reader is correctly connected to the kiosk via USB. It is also assumed that the kiosk system can successfully read and interpret the encoded data from the card and that the backend services are available to process authentication requests. Additionally, the system assumes that the card reader hardware is properly configured and maintained to ensure accurate and consistent readings.

7.4.2 RESPONSIBILITIES

The card swipe reader is responsible for capturing and transmitting user identification data from the magnetic stripe of a UTA ID card to the kiosk system for authentication. It must reliably read card data and ensure accurate input to the backend for processing. The system must securely handle this data

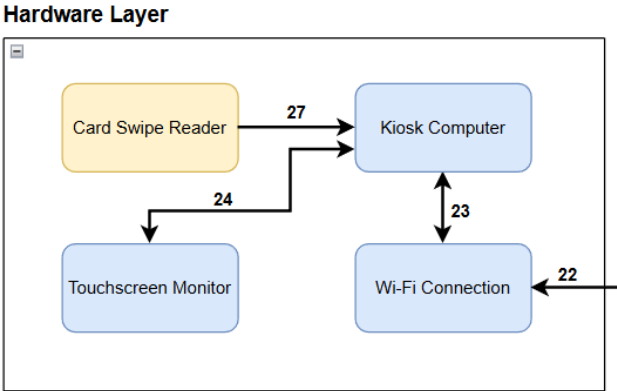


Figure 25: Card Swipe Reader Subsystem

during transmission and integrate it with the authentication process to grant or deny access to restricted features. It is also responsible for providing a convenient and efficient alternative to manual login while maintaining system security and reliability.

7.4.3 SUBSYSTEM INTERFACES

Table 21: Subsystem interfaces for Card Swiper Reader

ID	Description	Inputs	Outputs
#27	USB connection from the card swiper reader to the Kiosk Computer	User manually swipes UTA ID card	Magnetic Strip Data

REFERENCES